MODÈLES · CALCUL · MÉMOIRE · PREUVES

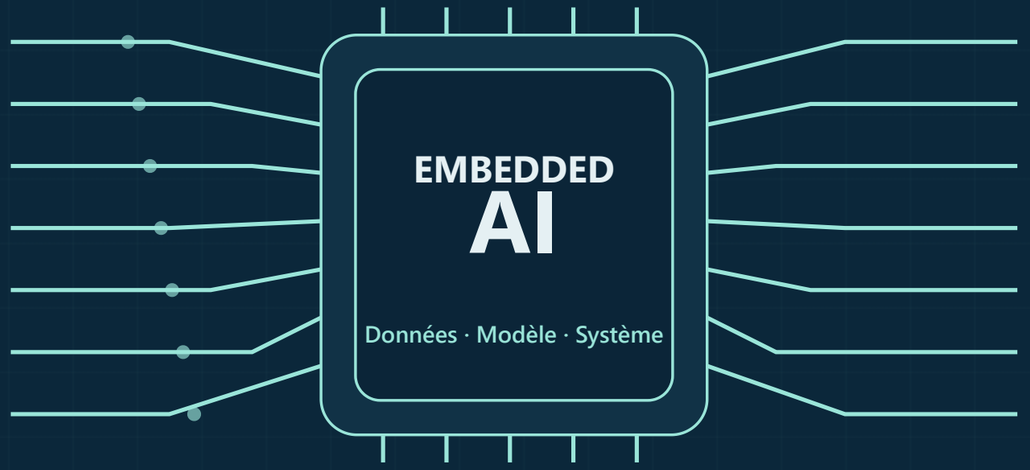
Faire tenir l'IA dans le monde réel.

IA pour les systèmes embarqués · Cours de synthèse · 3 heures

Roland Cédric TAYO

Futurs ingénieurs en systèmes embarqués

Sources vérifiées au 20 octobre 2026



Le service réel fixe le budget.

Cinq questions pour guider nos choix.

Du besoin au résultat mesuré, sans confondre démonstration et produit.

Pourquoi ici ? Choisir ce qui doit être calculé à bord.

Qu'est-ce qui coûte ? Lire les briques et le matériel.

Quelle preuve ? Optimiser, tester et expliquer les limites.

01 **Besoin & histoire**
27 min

02 **Briques du modèle**
40 min

03 **Budgets de la cible**
30 min + pause 10 min

04 **Optimisation**
32 min

05 **Déploiement & recherche**
41 min

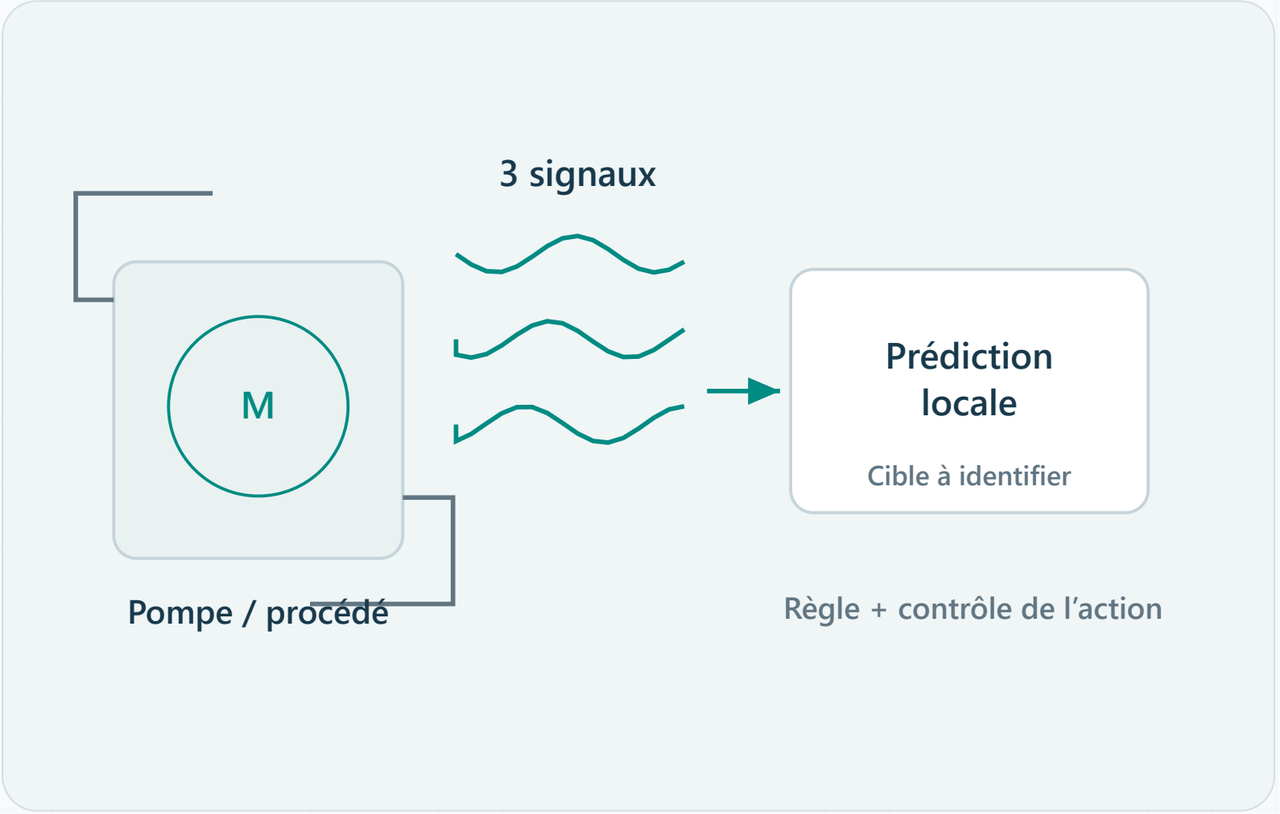
Une pompe doit-elle envoyer toutes ses mesures au cloud ?

Fil conducteur fictif : estimer une mesure future et signaler un comportement inhabituel.

Besoin : fonctionner quand le réseau est indisponible.

Contrainte : décider à temps, avec une petite mémoire.

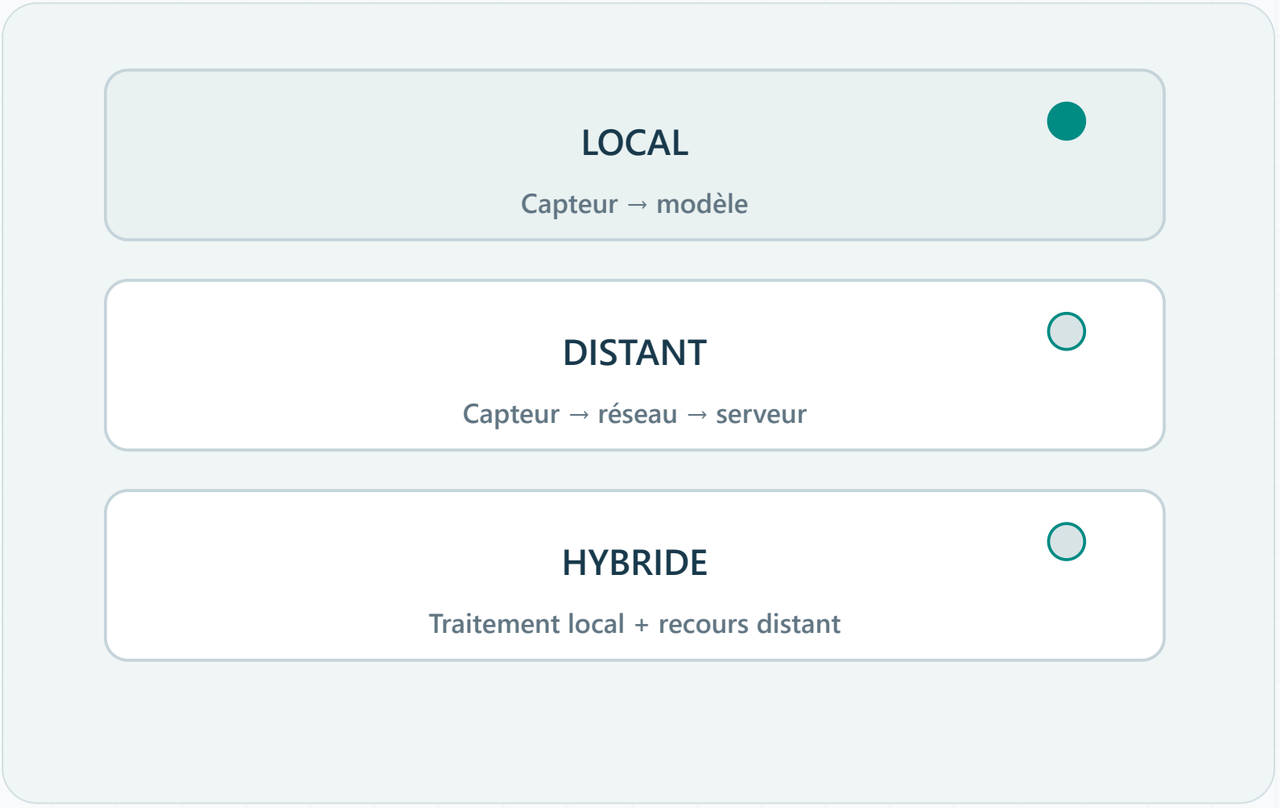
Question préalable : une règle ou un modèle linéaire suffit-il ?



Local, distant ou hybride : un choix d'architecture.

Rapprocher le calcul des données peut aider. Il faut encore compter le coût total.

- **Local :** autonomie réseau ; ressources à bord limitées.
- **Distant :** calcul disponible ; transmission et dépendance réseau.
- **Hybride :** filtrage local, assistance distante et mode dégradé défini.



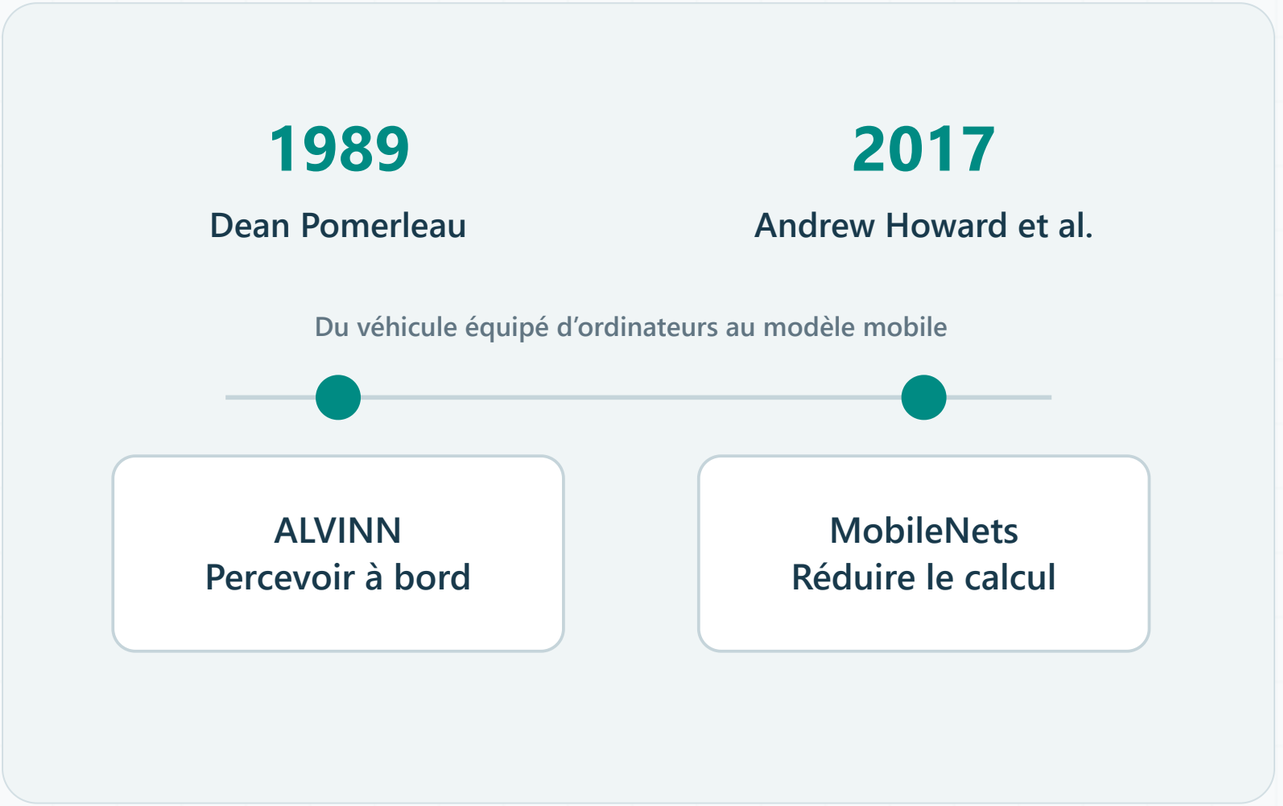
Le problème n’a pas commencé avec les grands modèles.

Deux étapes : apprendre à percevoir à bord, puis rendre le réseau plus économique.

1989 · Dean Pomerleau, ALVINN : relier perception et direction d’un véhicule.

2017 · Andrew Howard et al., MobileNets : réduire le calcul de la vision mobile.

La question demeure : que peut-on observer, calculer et décider sur la cible ?



2020–2026 : optimiser le système avec le modèle.

La nouveauté se joue aussi dans la mémoire, les noyaux et les instructions.

2020 · Lin, Chen et al. : MCUNet associe recherche de réseau et moteur MCU.

2024–2025 : architectures mobiles, quantification et noyaux bas bits.

2026 : adaptation matérielle et validation de très petites inférences.



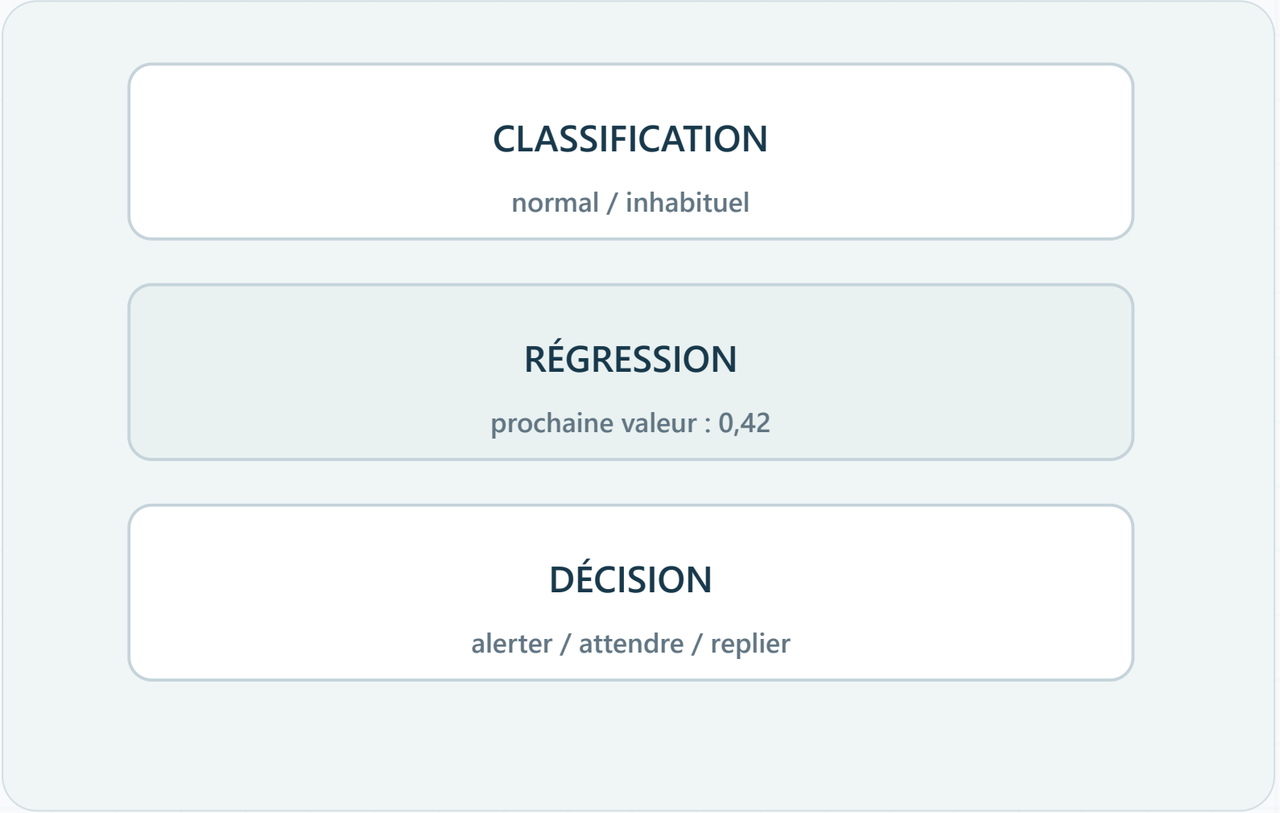
Choisir le problème avant de choisir le réseau.

Un modèle apprend une relation. Il ne remplace pas la définition du besoin.

Classer : reconnaître un état ou un mot.

Prédire / détecter : estimer une valeur ou repérer un écart.

Décider : traduire une sortie en action avec des règles et des limites.



Question 1 · Que faut-il garantir ?

Le réseau devient indisponible pendant dix minutes. Le système doit continuer à surveiller la pompe.

?

Discutez.
Choisissez.
Justifiez.

La bonne réponse doit rester vraie dans les conditions énoncées.

- **A** Choisir obligatoirement le réseau le plus précis du cloud.
- **B** Définir une surveillance locale et son comportement de repli.
- **C** Réduire le nombre de paramètres, puis ignorer la liaison.

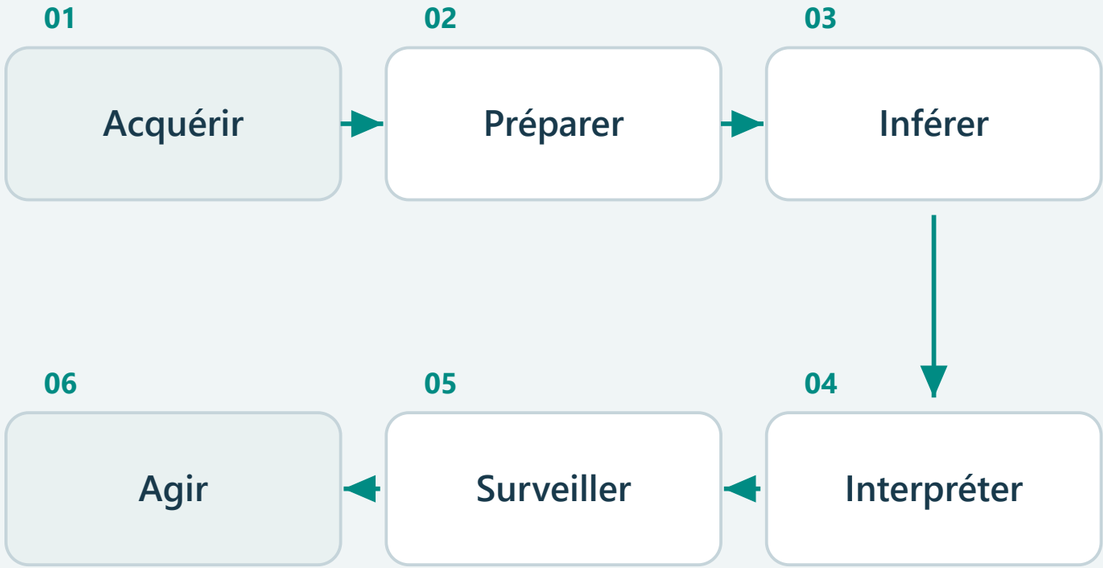
Une prédiction traverse une chaîne entière.

Le modèle n’est qu’une partie du temps et de la mémoire consommés.

> **Entrée :** échantillonner, synchroniser et vérifier.

> **Calcul :** préparer les données puis exécuter le graphe.

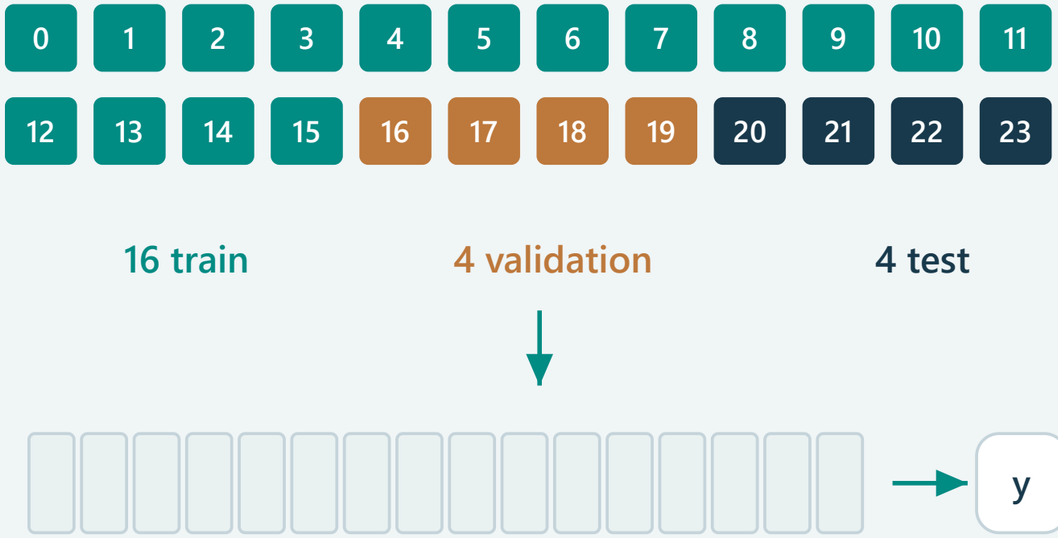
> **Sortie :** interpréter, surveiller et agir.



Brique 1 · Les données définissent ce que l'on apprend.

Séparer les séquences avant de fabriquer les fenêtres.

- **Apprentissage :** ajuster poids et normalisation.
- **Validation :** choisir les réglages.
- **Test réservé :** évaluer après avoir figé les choix.



Une fenêtre reste dans une seule séquence.

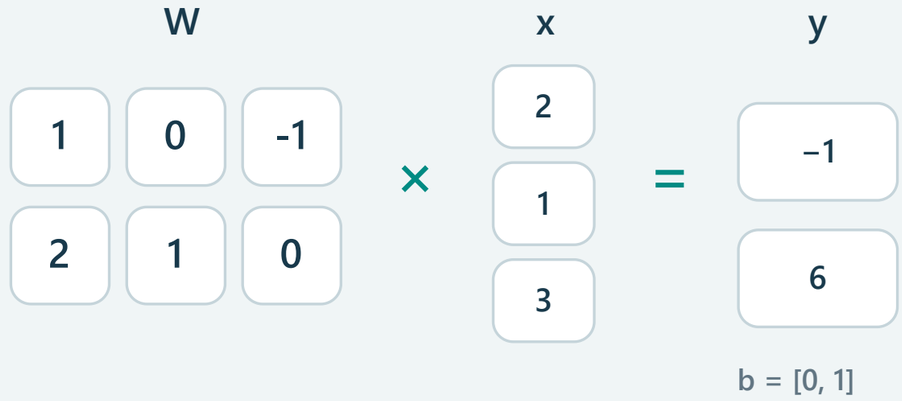
Brique 2 · Le produit dense : multiplier, puis additionner.

Un même calcul revient dans les petits réseaux, les convolutions et les Transformers.

> **Poids W :** coefficients appris ; **entrée x :** observations.

> **Biais b :** décalage ajouté à chaque sortie.

> **Coût :** m × n MAC pour n entrées et m sorties.



$$y_1 = 1\times2 + 0\times1 - 1\times3 + 0 = -1$$

6 MAC + 2 additions de biais

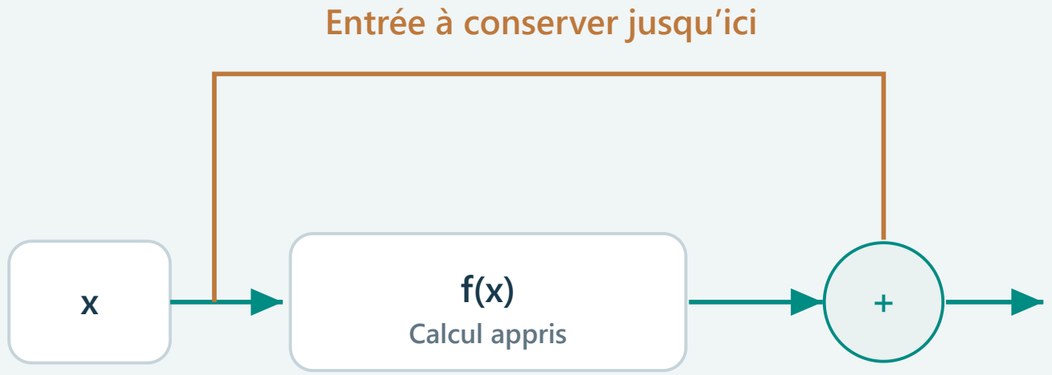
Brique 3 · Transformer et conserver l'information.

Activation, connexion résiduelle et normalisation ont des rôles différents.

ReLU : max(0, x), pour introduire une non-linéarité.

Résidu : x + f(x), pour ajouter une correction apprise.

Normalisation : stabiliser une échelle ; son coût dépend de sa forme.



$$x + f(x)$$

Le raccourci prolonge la vie de x.

Brique 4 · La convolution réutilise un petit motif.

Une convolution séparée découpe le filtrage spatial et le mélange des canaux.

Classique : chaque sortie combine espace et canaux.

Depthwise + 1×1 : filtrer chaque canal, puis les mélanger.

À vérifier : moins de MAC peut encore produire plus de transferts.



Calcul par position · exemple arithmétique, pas un benchmark

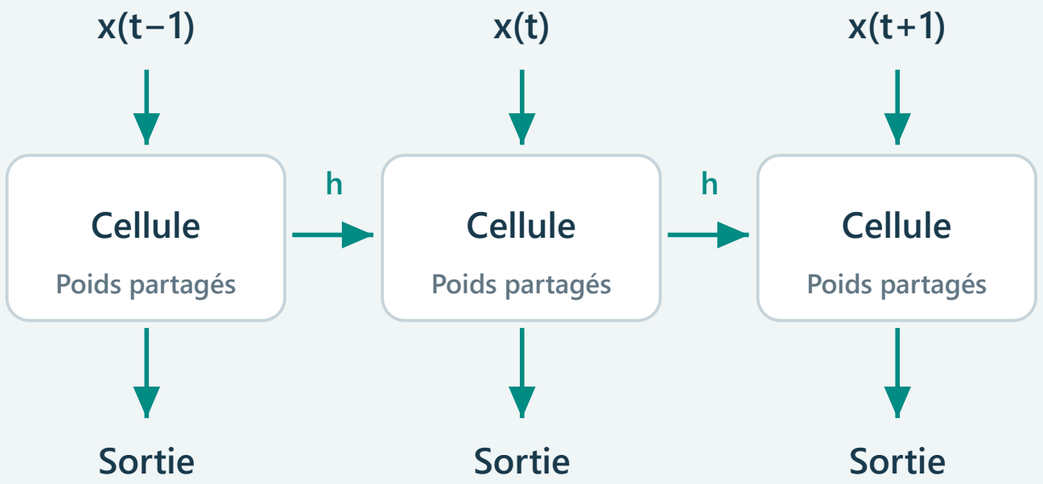
Brique 5 · Un état mémorise une partie du passé.

Un réseau récurrent met à jour un petit état à chaque échantillon.

État h : résumé réutilisé au pas suivant.

Intérêt : traiter un flux sans garder tout son historique.

Limite : dépendance séquentielle, dérive et temps de mise en régime.



État transmis ≠ historique complet stocké

Brique 6 · L’attention choisit quelles relations regarder.

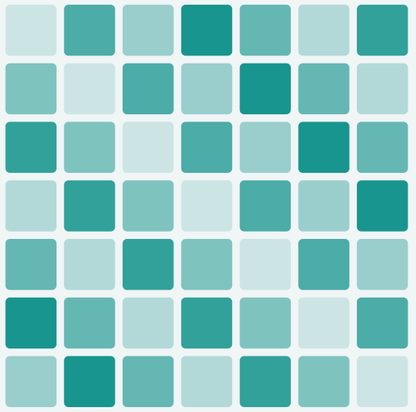
Au lieu d’un mélange fixe, les relations dépendent du contenu de l’entrée.

Q, K, V : requêtes, clés et valeurs calculées par projections.

Comparer : associer chaque position à d’autres positions.

Coût : une matrice naïve de scores grandit comme le carré de la longueur.

Scores entre positions



Normaliser
puis mélanger

$\text{softmax}(QK^T / \sqrt{d})\,V$

^{T} : transpose · d : dimension d’une tête

Doubler la longueur → 4× plus de scores naïfs

Carte d’attention illustrative ; aucune sortie de modèle mesurée

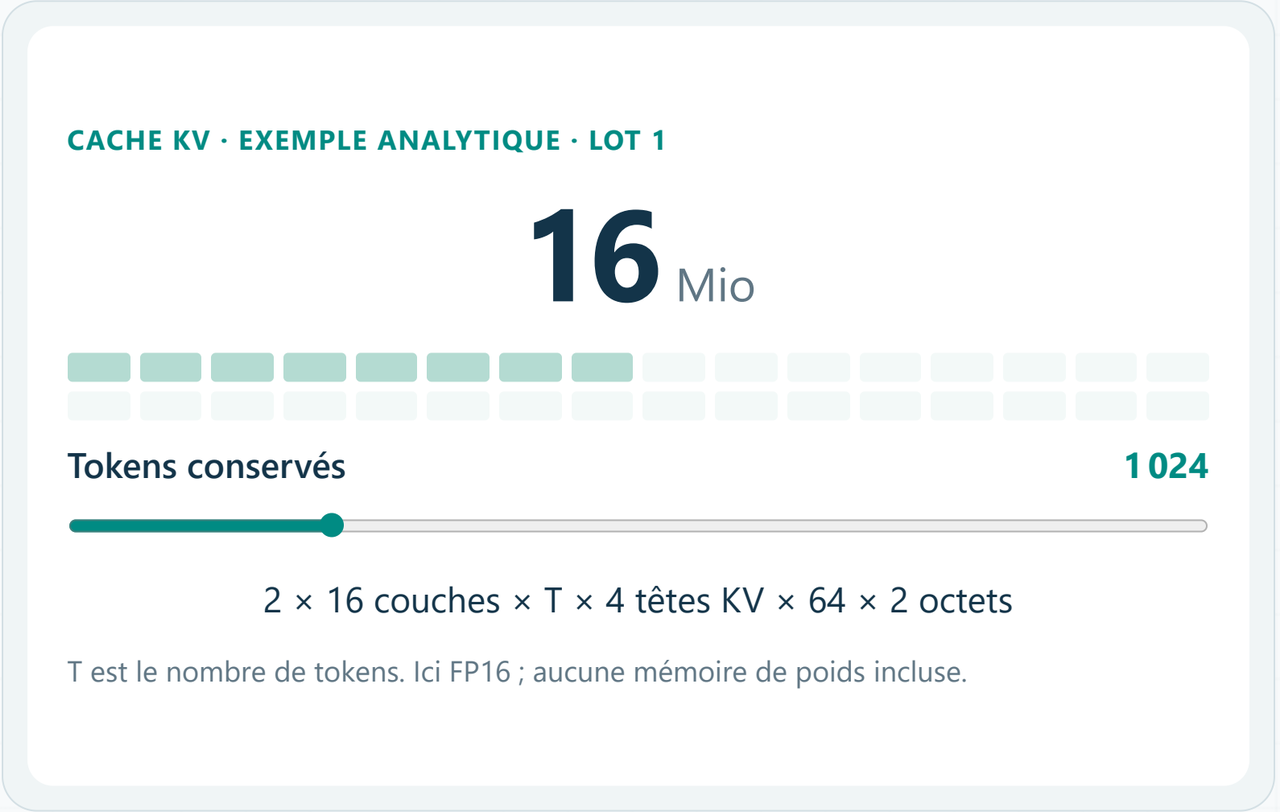
Brique 7 · Le contexte consomme aussi de la mémoire.

Un cache KV évite de recalculer d'anciennes clés et valeurs lors de la génération.

Préremplissage : traiter la consigne initiale.

Décodage : produire ensuite un token à la fois.

Conséquence : réduire les poids ne réduit pas automatiquement le cache.



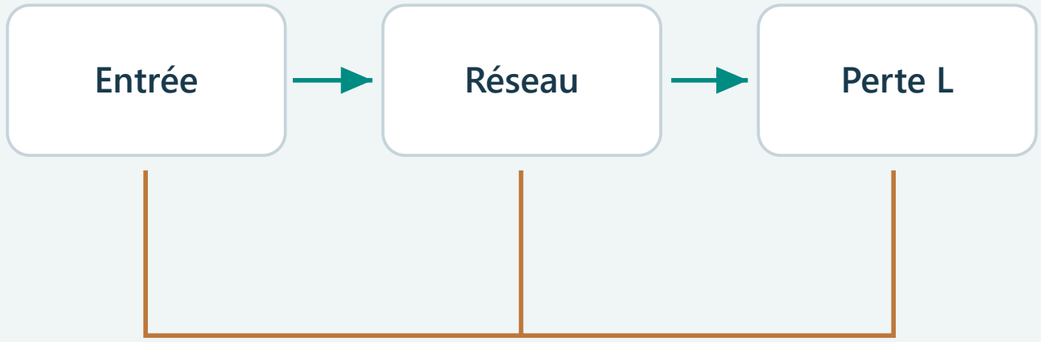
Brique 8 · Apprendre demande plus que prédire.

L'inférence lit les poids ; l'apprentissage calcule aussi comment les changer.

Perte : mesurer l'erreur entre prédiction et cible.

Gradient : calculer la sensibilité de cette erreur aux poids.

Mise à jour : corriger les poids avec un petit pas.



Gradients ∇L : parcours arrière

$$\theta \leftarrow \theta - \eta \nabla L$$

θ : paramètres · η : pas · ∇L : gradient de la perte

Question 2 · « 4 bits » veut-il dire « tout petit » ?

Les poids passent de 16 à 4 bits. Le cache KV, les activations et le runtime sont inchangés.

?

Discutez.
Choisissez.
Justifiez.

La bonne réponse doit rester vraie dans les conditions énoncées.

A Le stockage brut des poids est divisé par quatre ; le reste doit être recompté.

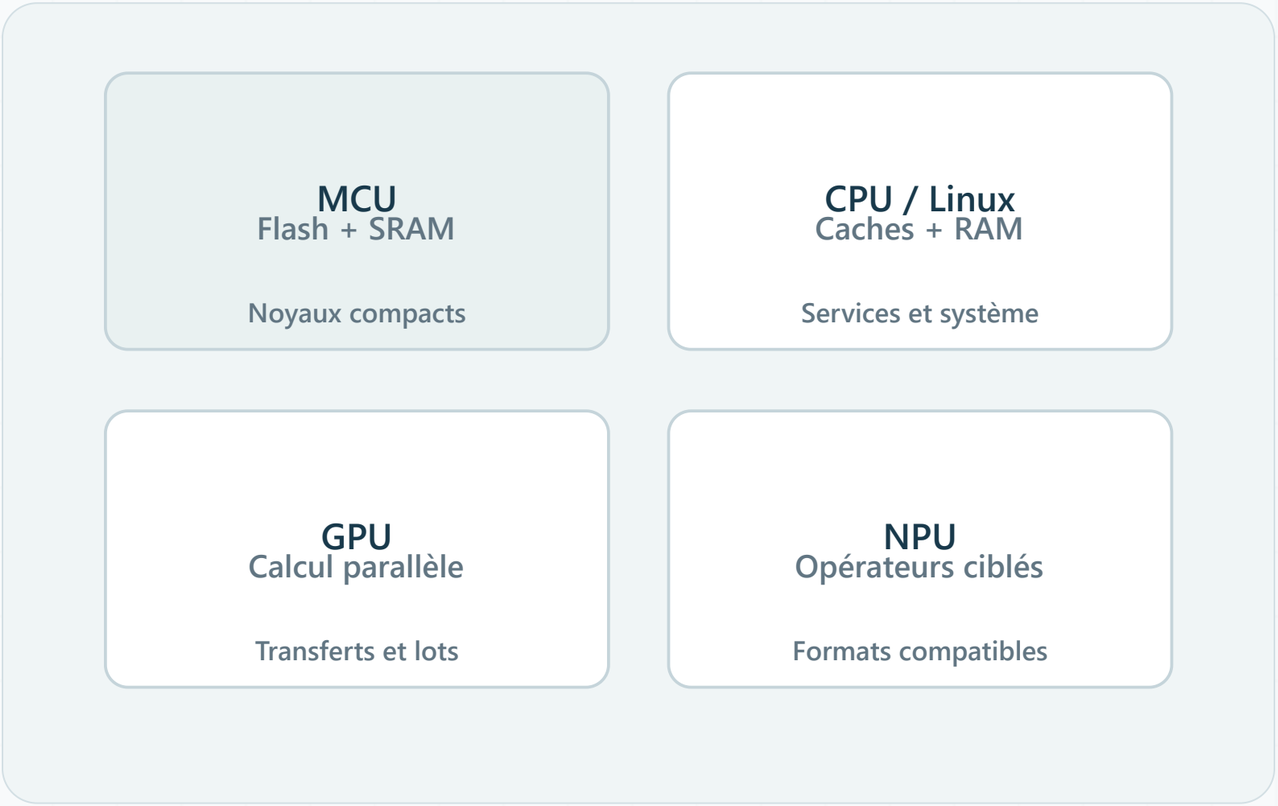
B Toute la RAM est forcément divisée par quatre.

C Le modèle produit forcément quatre fois plus de sorties.

Un MCU, un Pi et un NPU ne posent pas le même problème.

Choisir une famille de cible avant de recopier un résultat de papier.

- **MCU :** petite SRAM, Flash, souvent sans système Linux.
- **Processeur Linux :** RAM plus grande, caches, services et ordonnancement.
- **GPU / NPU :** parallélisme spécialisé, formats et opérations imposés.



La mémoire a plusieurs adresses et plusieurs durées de vie.

Poids stockés, valeurs intermédiaires et pic de travail sont trois questions.

Flash : programme et constantes conservés hors alimentation.

SRAM / RAM : buffers, pile, données en cours de calcul.

Pic : somme des objets vivants au même instant, pas de tous les objets du graphe.

EXEMPLE MCU · POIDS CONSERVÉS EN FLASH

Poids bruts	Travail SRAM
400 Ko	**120 Ko**
100 000 paramètres	activations + buffers + pile



Poids FP32 | Poids INT8 seuls

Budgets fictifs distincts. Échelles et métadonnées exclues du payload ; SRAM inchangée dans cet exemple.

Le calcul peut attendre la mémoire.

Compter les opérations ne suffit pas : il faut les alimenter en données.

> **Réemploi :** utiliser plusieurs fois une donnée déjà proche du cœur.

> **Localité :** parcourir les adresses dans un ordre favorable.

> **Borne utile :** comparer temps de calcul et temps de transfert.

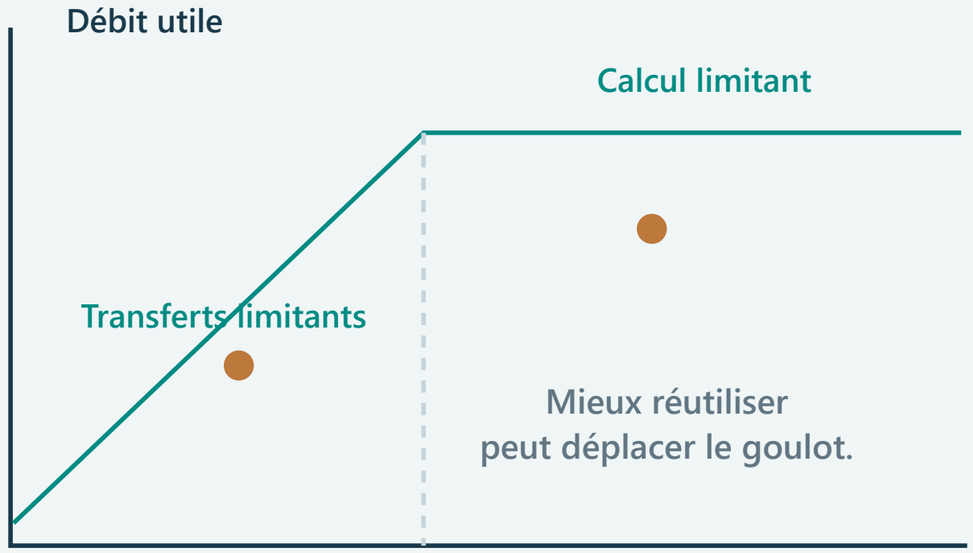


Schéma conceptuel · axes non calibrés

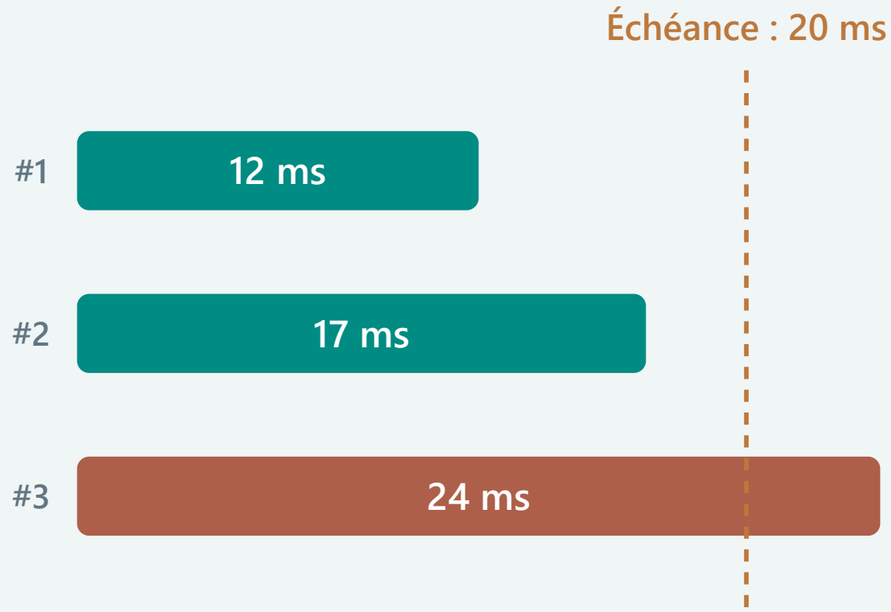
Un temps moyen ne garantit pas une échéance.

La sortie doit être disponible à temps, même quand le système est occupé.

> **Latence :** délai pour une entrée ; **débit :** sorties par seconde.

> **Gigue :** variation de ce délai entre les appels.

> **Temps réel :** définir l'échéance, la surcharge et le repli.



Durées pédagogiques · le troisième appel est trop tardif.

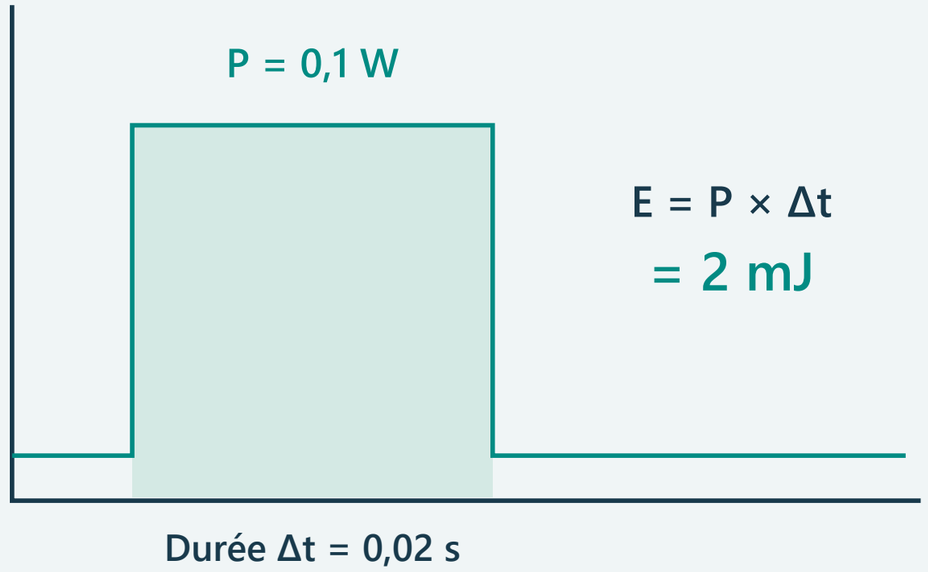
Énergie, puissance et autonomie : trois grandeurs différentes.

Un calcul plus rapide peut aider ; la mesure doit porter sur le bon périmètre.

Puissance P : consommation instantanée, en watts.

Énergie E : consommation sur une durée, en joules.

Système : intégrer capteur, veille, calcul, radio et alimentation.



Exemple à puissance constante · pas une mesure de carte

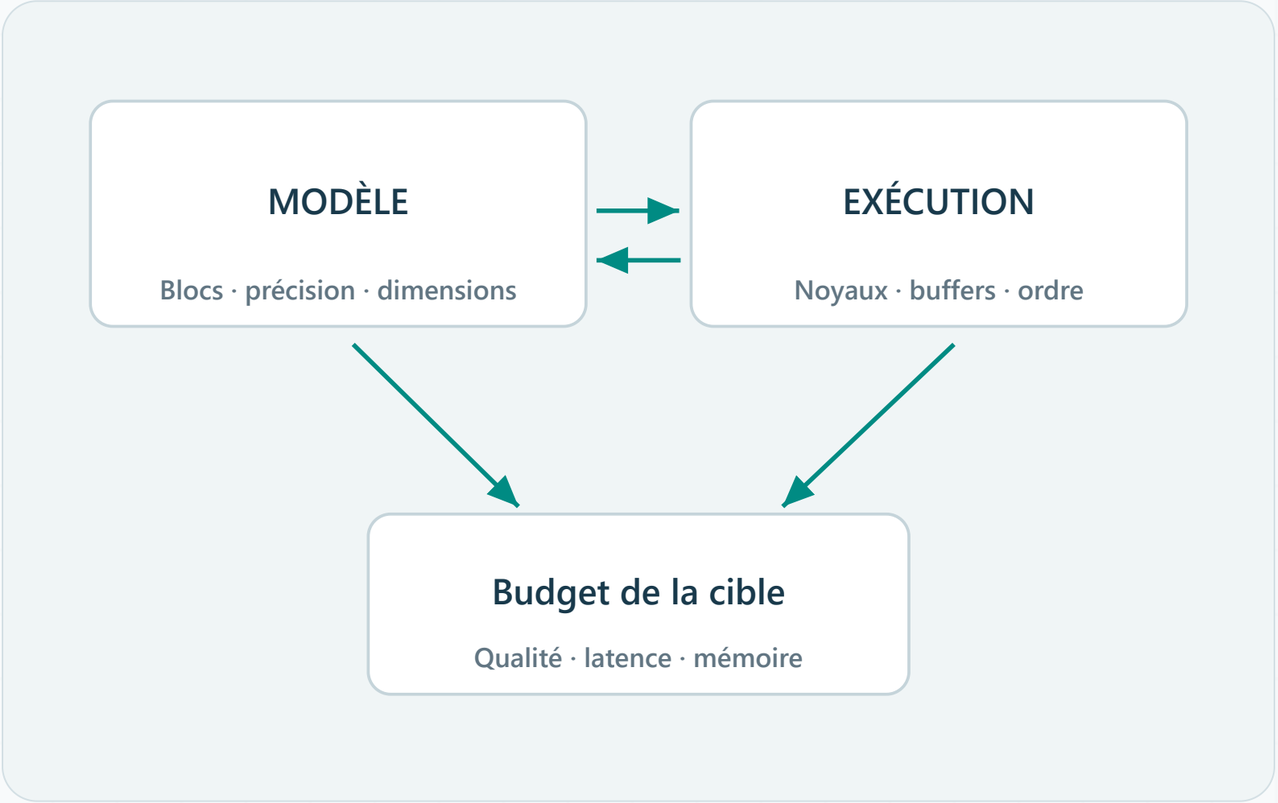
Deux recherches : déplacer la contrainte au bon endroit.

Architecture et exécution se répondent ; une seule métrique masque le compromis.

MCUNetV2 · 2021 : calculer par morceaux pour réduire le pic mémoire.

MobileNetV4 · 2024 : concevoir des blocs efficaces pour plusieurs cibles mobiles.

Question ouverte : le gain reste-t-il vrai sur notre cible et notre tâche ?



Question 3 · Le modèle tient-il ?

Une carte a 128 Kio de SRAM. Le modèle utilise 80 Kio de constantes placées en Flash.

?

Discutez.
Choisissez.
Justifiez.

La bonne réponse doit rester vraie dans les conditions énoncées.

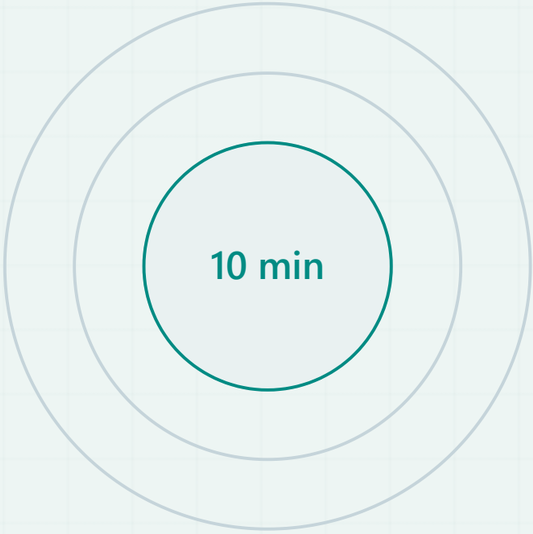
- **A** Oui : 80 est inférieur à 128.
- **B** Non : tous les poids doivent toujours être copiés en SRAM.
- **C** Impossible de conclure sans activations, buffers, pile et placement réel.

Pause · 10 minutes.

Au retour : choisir une optimisation, puis chercher la preuve de son intérêt.

> Avant la pause, retenez : **qualité + mémoire + temps + énergie.**

> Une opération rapide seule peut laisser le système lent.

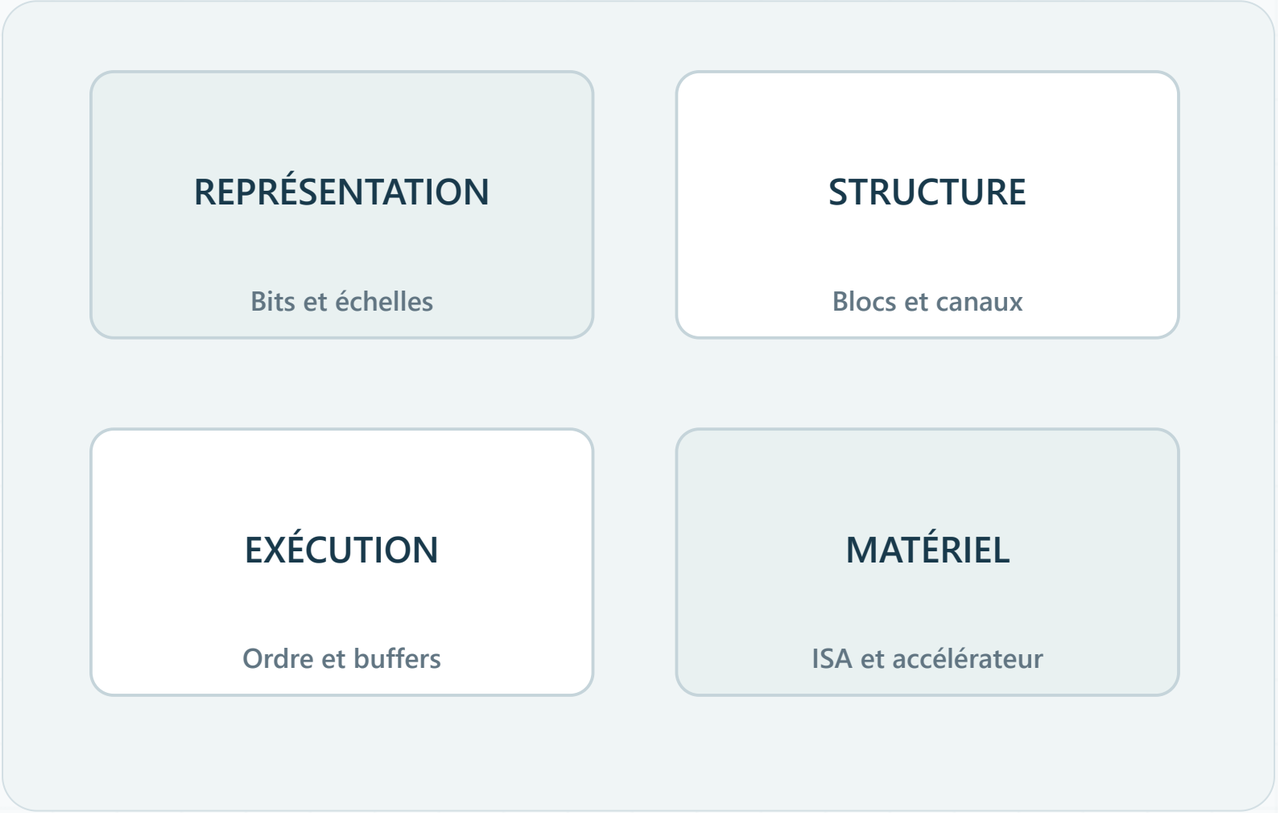


Le modèle et son matériel forment un système.

Quatre leviers, quatre façons de se tromper.

Commencer par le goulot observé, pas par la technique à la mode.

- **Représentation :** réduire la précision utile.
- **Modèle :** réduire largeur, blocs ou structure.
- **Exécution et matériel :** réutiliser, fusionner et choisir les bons noyaux.



Quantifier : garder moins de chiffres utiles.

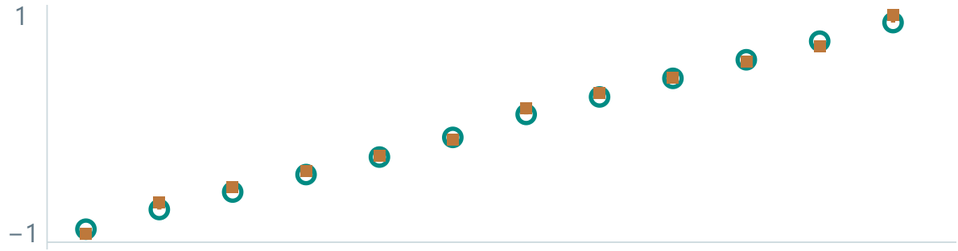
Le format réduit un coût, au prix d'une approximation à vérifier.

> **Échelle s :** taille d'un pas entre deux valeurs représentables.

> **Code q :** entier arrondi et limité à une plage.

> **Attention :** poids, activations et accumulateurs peuvent avoir des formats différents.

ARRONDI SYMÉTRIQUE · EXEMPLE SUR [−1, 1]



12 valeurs illustratives

Précision des codes **4 bits**

Pas s : **0,1429**

Erreur max illustrée : **0,07**

Cercles : valeurs initiales ; carrés : valeurs reconstruites. Pas un test de qualité de réseau.

Calibrer, entraîner, puis comparer le vrai calcul.

La meilleure précision n'est pas forcément uniforme dans tout le réseau.

PTQ : quantifier après entraînement, avec calibration si nécessaire.

QAT : simuler les effets de quantification pendant l'apprentissage.

AWQ · 2024 : protéger les canaux sensibles à partir des activations.



Préserver ce qui change le plus la sortie

Puis vérifier qualité et noyau réellement exécuté.

Supprimer, simplifier, ou apprendre à imiter.

Compresser un fichier ne suffit pas à accélérer son exécution.

Élagage : retirer poids ou canaux ; exploiter réellement la structure obtenue.

Distillation : entraîner un petit élève à reproduire un enseignant.

Choix d'architecture : chercher un compromis compatible avec la cible.



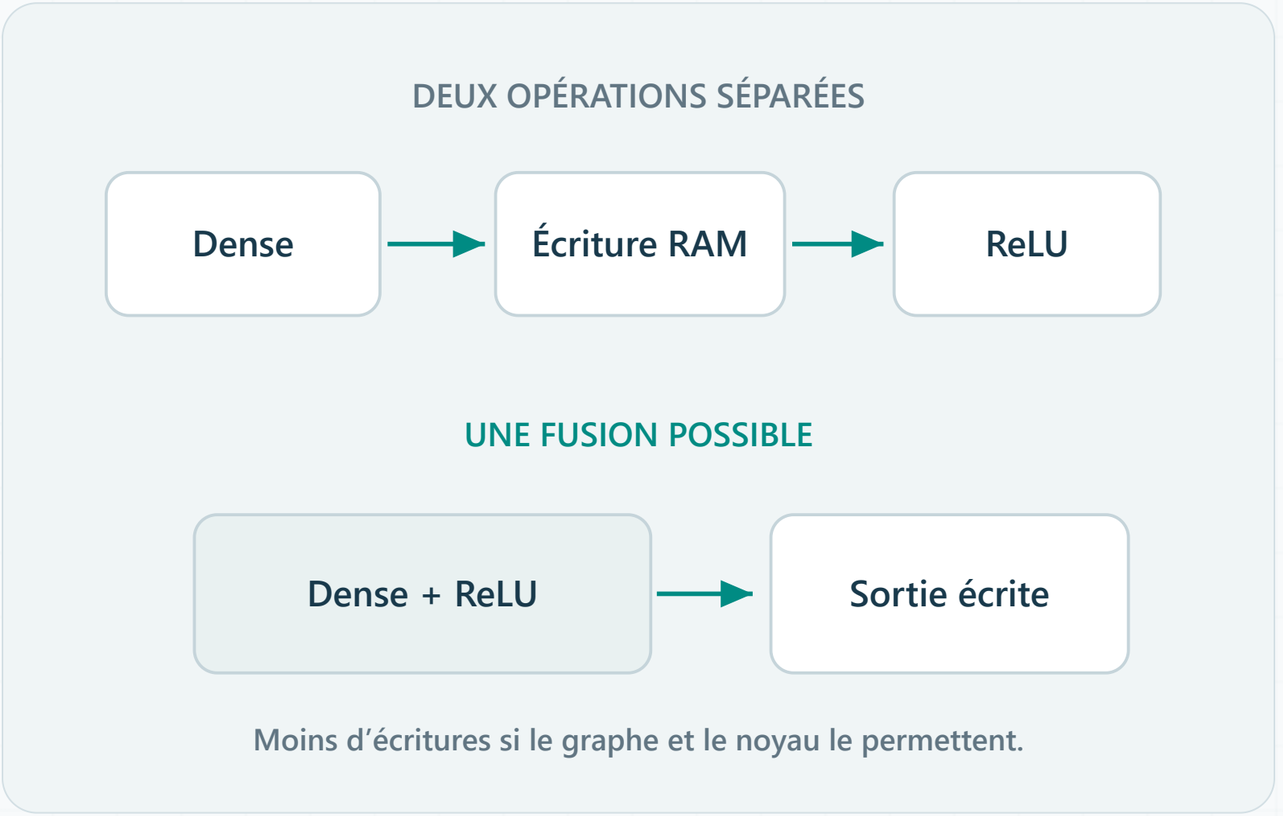
Le noyau doit exploiter la structure restante.

Distillation : enseignant → petit élève à entraîner

Réorganiser sans changer la fonction mathématique.

Une optimisation de brique doit rester correcte une fois réintégrée.

- **Réemploi :** traiter plusieurs sorties avec les mêmes entrées.
- **Fusion :** éviter une écriture intermédiaire lorsque c’est possible.
- **Coût caché :** registres, conversions, code et buffers temporaires.



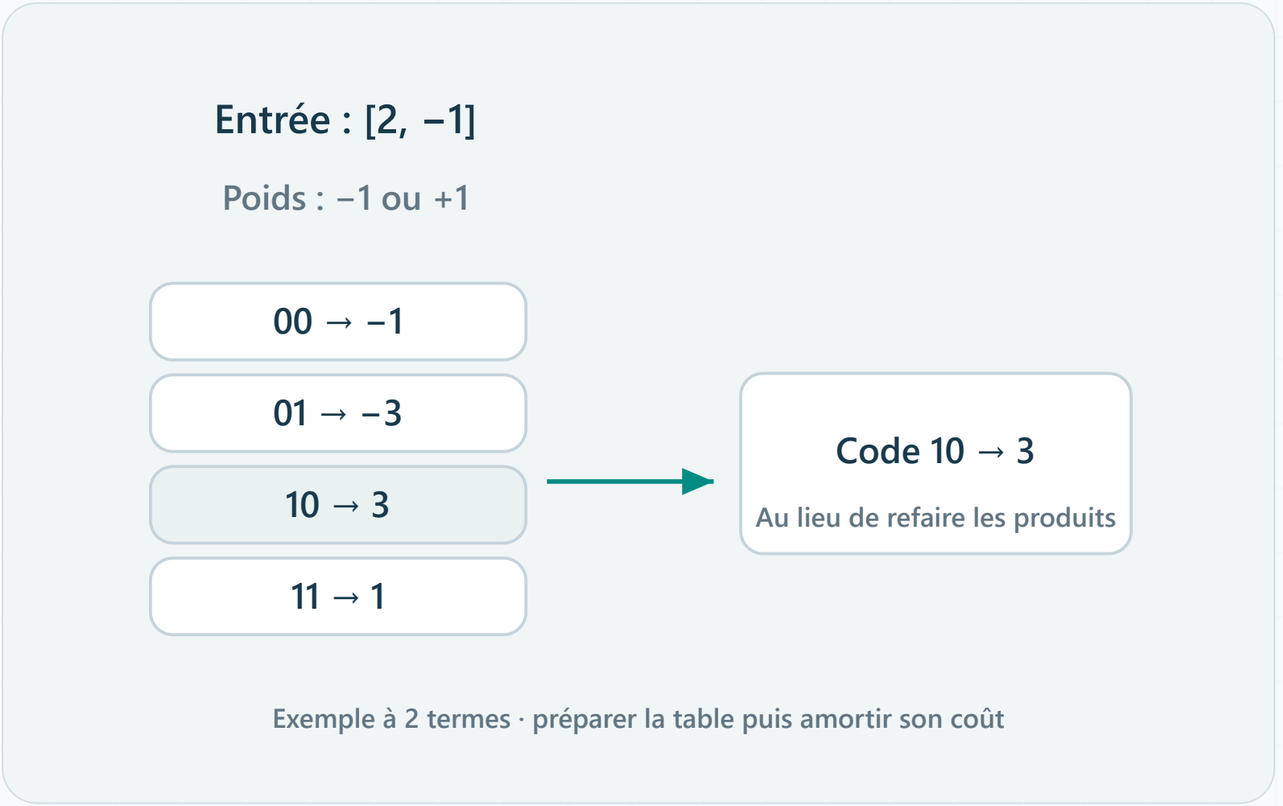
T-MAC · Remplacer un calcul par une consultation de table.

Jianyu Wei et al. · EuroSys 2025 · Une idée système pour produits bas bits sur CPU.

> **Problème :** les poids bas bits peuvent coûter cher à déquantifier.

> **Idée :** préparer des combinaisons, puis les retrouver par indices.

> **Limite :** la table et ses accès doivent être moins coûteux que le calcul évité.



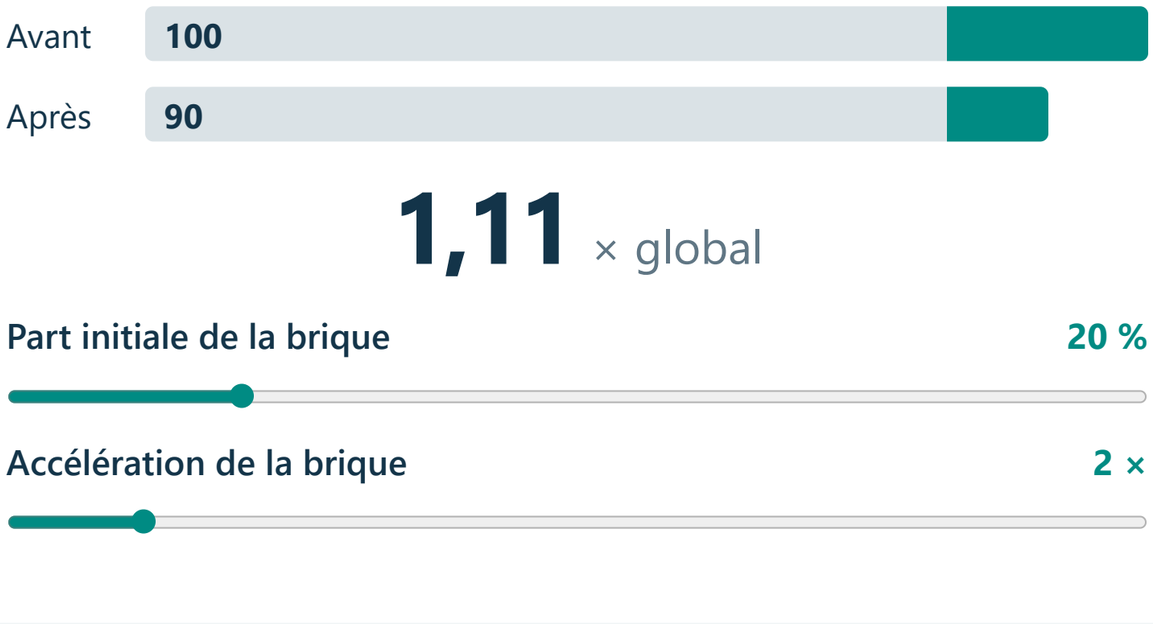
Accélérer une brique ne suffit pas à accélérer le système.

La partie inchangée impose une limite au gain global.

p : fraction du temps initial passée dans la brique.

s : accélération de cette seule brique.

Gain global : 1 / [(1 − p) + p/s], si le reste est inchangé.



Question 4 · Quel résultat peut-on défendre ?

Sur PC, la projection gagne 10 %. Le réseau entier ralentit de 5 %. La carte n’a pas été testée.

?

Discutez.
Choisissez.
Justifiez.

La bonne réponse doit rester vraie dans les conditions énoncées.

- **A** La brique gagne dans ce protocole PC, sans gain global ni preuve sur carte.
- **B** Il faut conserver seulement le meilleur chronométrage.
- **C** L’IA embarquée est accélérée de 10 %.

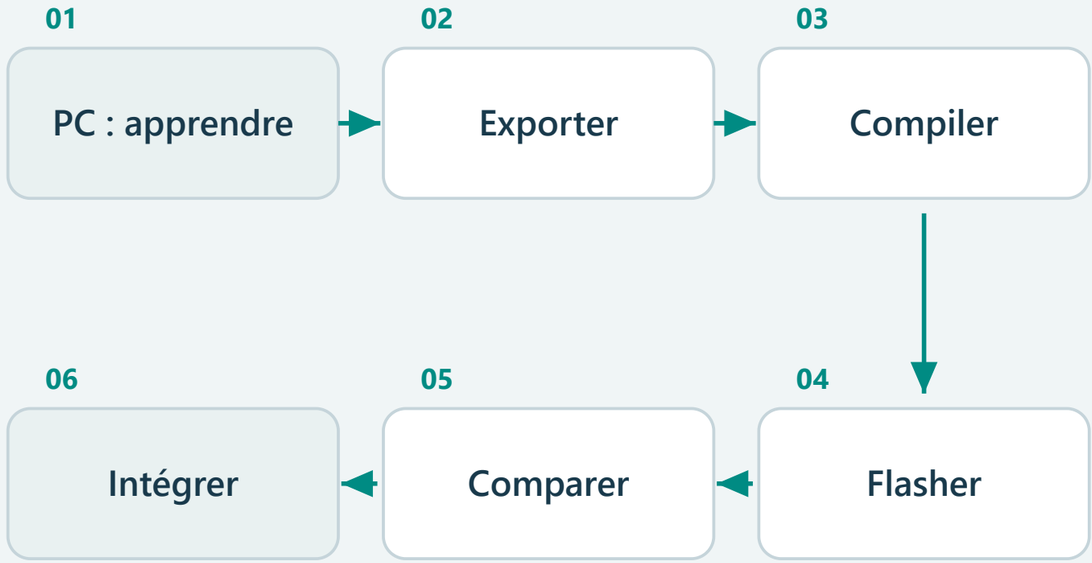
Déployer : exporter, compiler, vérifier, intégrer.

Un modèle qui s’ouvre dans un outil n’est pas encore une application validée.

Exporter : graphe, poids, formes et normalisation.

Vérifier : opérateurs, types, ordre des canaux et équivalence numérique.

Intégrer : entrées réelles, états, tâches, erreurs et mises à jour.



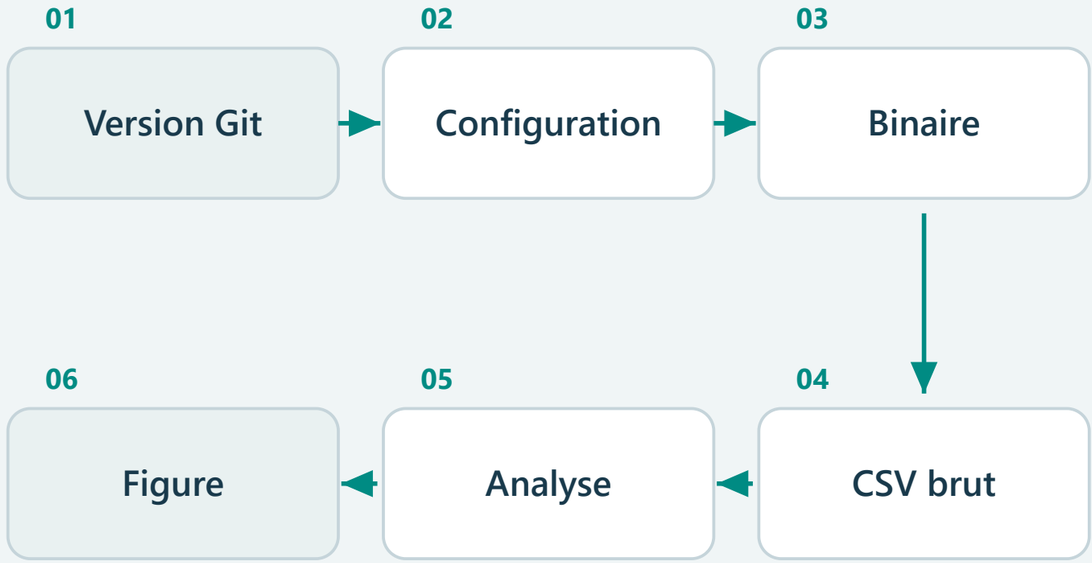
Mesurer pour qu'un autre ingénieur puisse vous contredire.

Un chiffre devient une preuve quand son origine est retrouvable.

Figer : données, poids, carte, compilateur, options et périmètre.

Mesurer : timer vérifié, répétitions, alternance et données brutes.

Relier : commit → binaire → CSV → figure → conclusion.



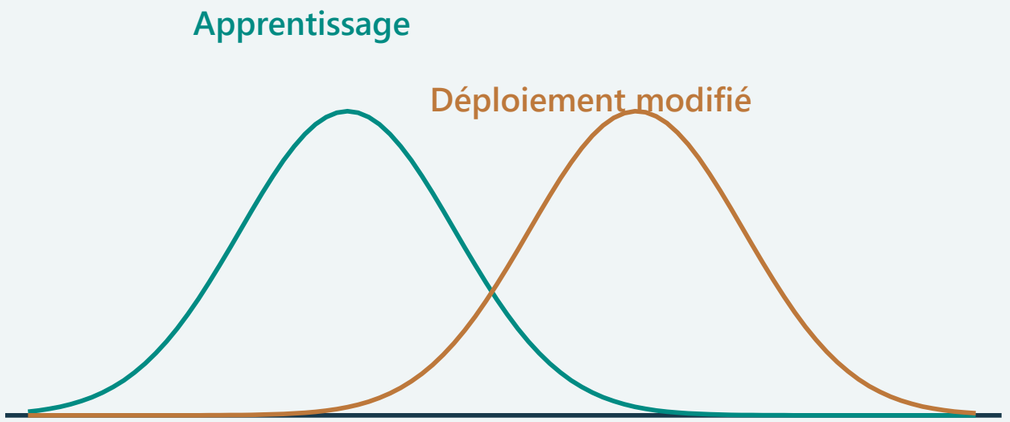
Une bonne moyenne peut masquer une panne.

La fiabilité concerne les entrées inattendues et les conséquences des erreurs.

Dérive : capteur vieilli, autre machine ou environnement différent.

Erreur : faux positif, faux négatif et coût associé.

Repli : signaler l'incertitude, limiter l'action ou revenir à une règle.



Un changement de distribution peut déplacer les erreurs.

Schéma qualitatif : aucune fréquence mesurée.

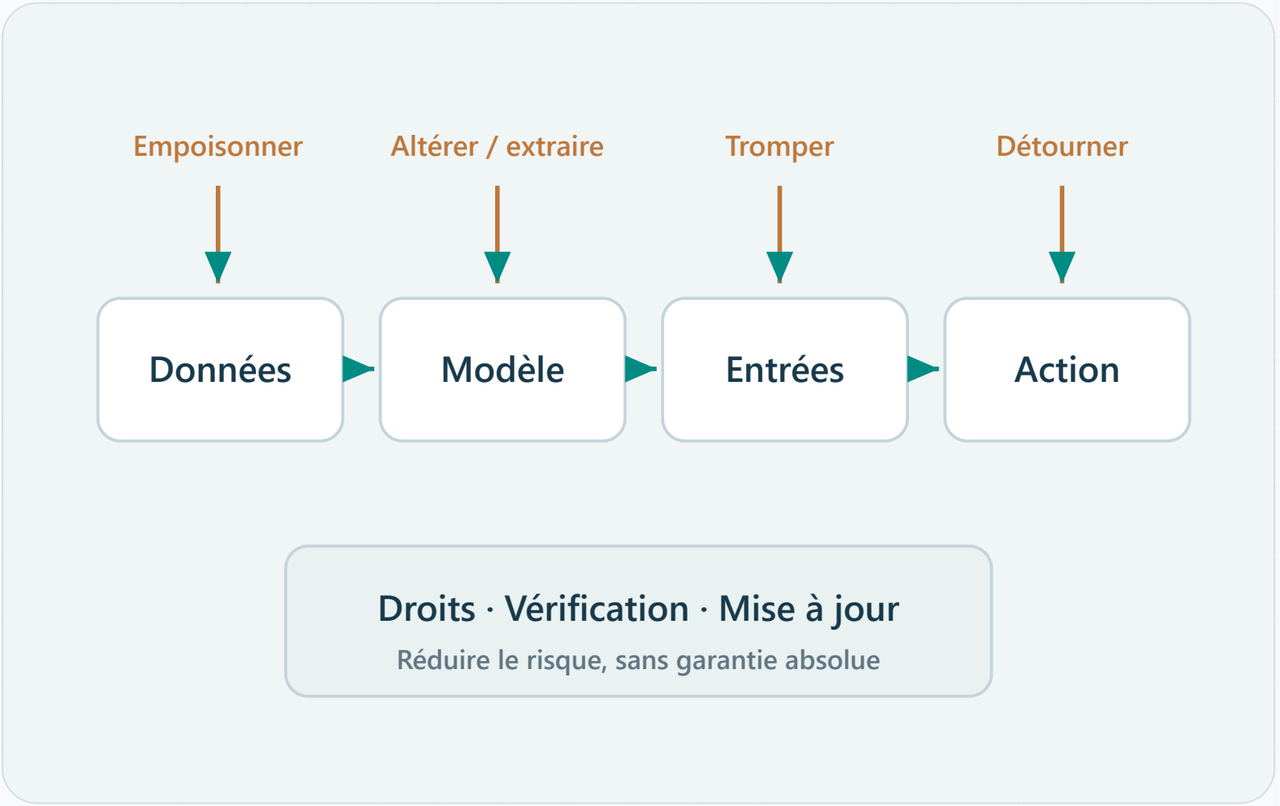
Local ne signifie pas confidentiel, robuste ou sûr.

L'IA ajoute des surfaces d'attaque à celles du logiciel embarqué.

Données / entrées : empoisonnement et perturbations adverses.

Modèle / logiciel : extraction, dépendances ou mises à jour altérées.

Action : contrôler les droits et valider les commandes proposées.



« Petit modèle de langage » ne signifie pas « modèle pour MCU ».

Le mot petit dépend du point de comparaison.

> **1,7 milliard de paramètres à 4 bits :** environ 850 Mo de poids bruts.

> **À ajouter :** métadonnées, activations, cache KV et moteur.

> **À décider :** besoin réel de génération, ou classification spécialisée ?



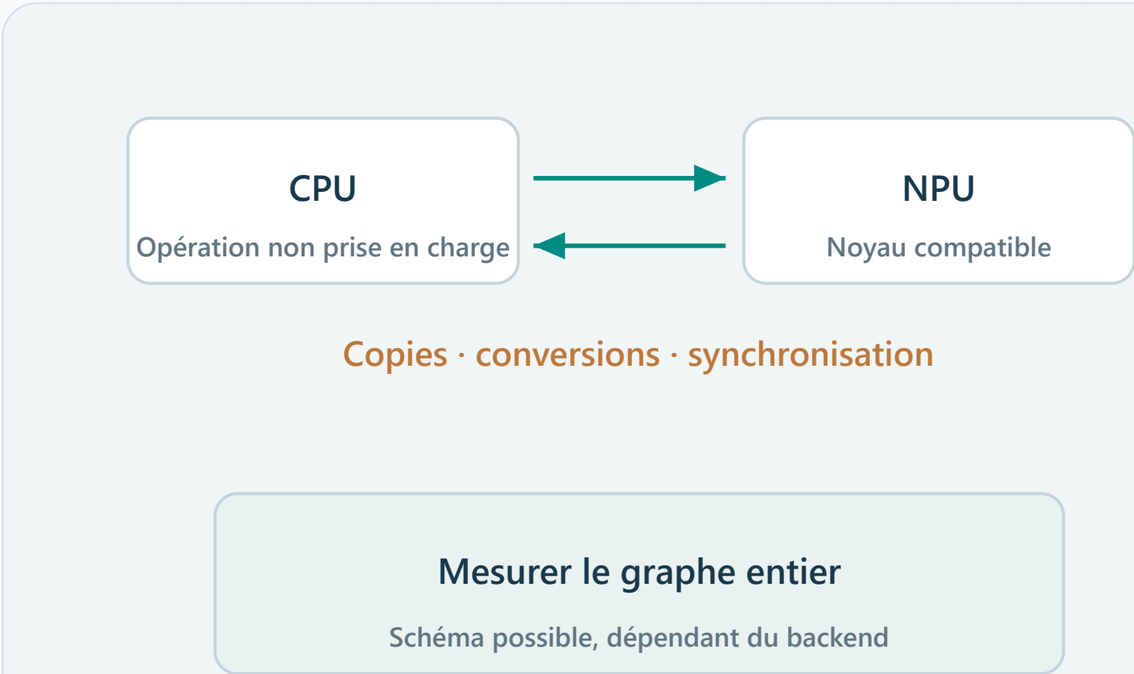
Veille 2025 · Le NPU ne gagne que si le graphe lui convient.

T-MAN · Wei, Li et al. · Prépublication : lire ce qui est accéléré, et ce qui ne l'est pas.

> **Problème :** des opérations périphériques limitent un accélérateur spécialisé.

> **Piste :** adapter tables, tuilage et pipeline.

> **À contrôler :** préremplissage, décodage, transferts et comparaison de départ.



Veille 2026 · Deux voies, deux niveaux d'intervention.

Des pistes actuelles à examiner ; aucune promesse universelle de performance.

FastGRNN · Kizilates : très petite récurrence, tables et validation sur MCU.

RISC-V · Kumar et al. : instructions et compilation pour Transformers compacts.

Regard critique : mise en régime, graines, mémoire, aire et protocole.

JUIN 2026

FastGRNN sur MCU

Mise en régime à examiner

Emre Can Kizilates

OCTOBRE 2026

Transformer sur RISC-V

Instructions + compilation
Aire et énergie à compter

Ajay Kumar M et al.

Deux prépublications : hypothèses et périmètres à vérifier.

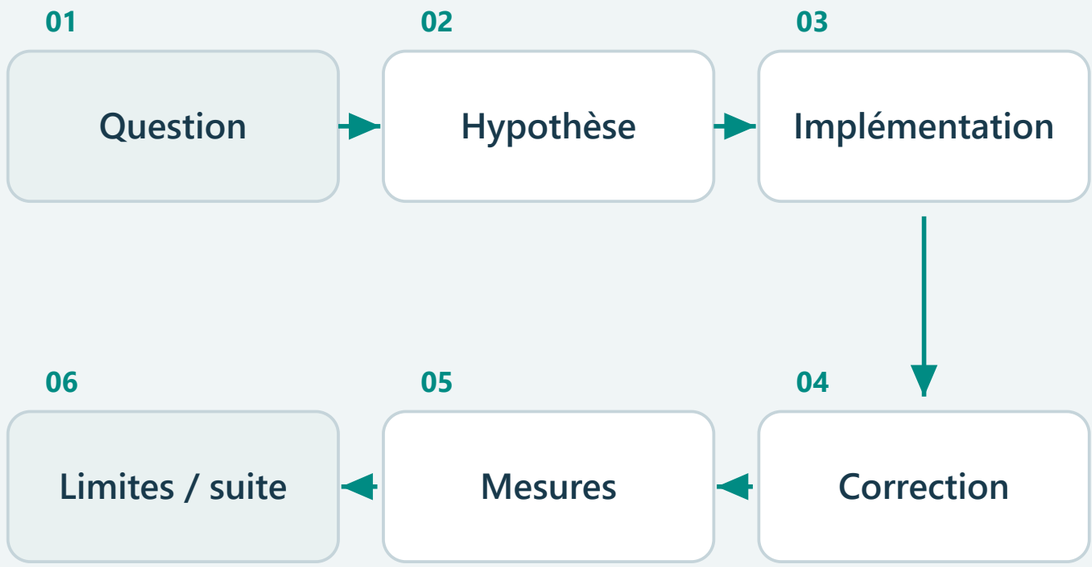
Notre TD : une petite question, une expérience défendable.

Le travail de recherche commence quand on accepte que l'hypothèse puisse échouer.

Construire : petit Mixer manuel, apprentissage et références.

Enquêter : une projection dense, une hypothèse, des variantes limitées.

Conclure : qualité, coûts PC puis carte, limites et expérience suivante.



La conclusion peut être négative.

Question 5 · Le réseau perd face au modèle linéaire.

Les données et le protocole sont corrects. La variante optimisée ralentit aussi le réseau sur carte.

?

Discutez.
Choisissez.
Justifiez.

La bonne réponse doit rester vraie dans les conditions énoncées.

- **A** Changer le test jusqu'à obtenir une victoire.
- **B** Masquer les références simples dans l'article.
- **C** Rapporter les résultats, expliquer les limites et proposer un nouveau test séparé.

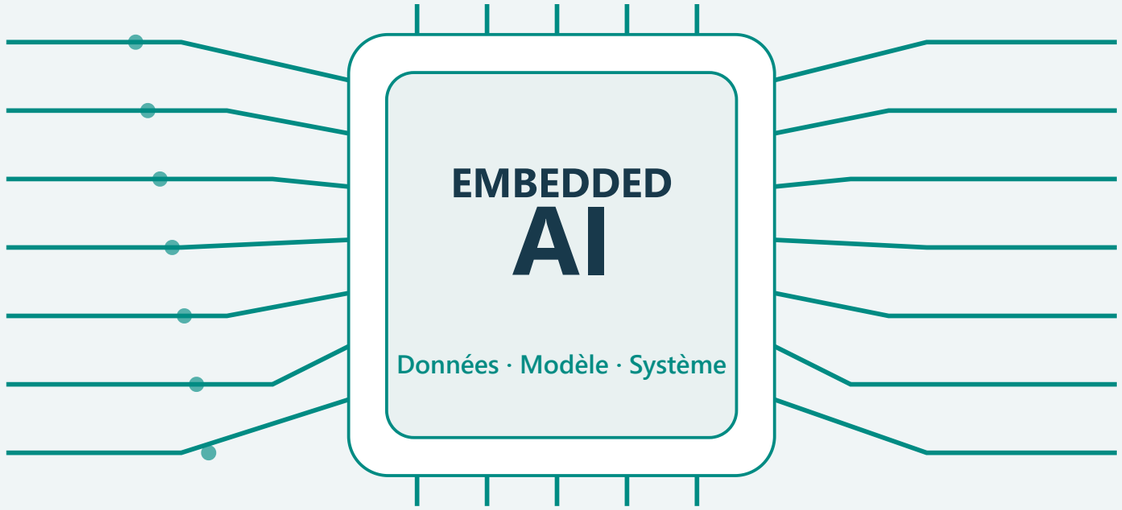
Votre métier : choisir un compromis et le prouver.

Le bon modèle est celui qui rend le service attendu dans le système réel.

> **Besoin clair.** Référence simple et données crédibles.

> **Budget explicite.** Matériel, mémoire, temps et énergie.

> **Preuve reproductible.** Tests, mesures, limites et maintien en service.



Le service réel fixe le budget.

Sources et limites de lecture · 1/6

Articles originaux, prépublications et documentation officielle · état de vérification : 20 octobre 2026.

R01 ALVINN: An Autonomous Land Vehicle In a Neural Network ↗
Dean Pomerleau · 1989 · Repère historique · publication CMU / actes NeurIPS
Relier perception apprise et action à bord d'un véhicule.

R02 MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications ↗
Andrew G. Howard et al. · 2017 · Article fondateur · arXiv 1704.04861
Séparer convolution spatiale et mélange des canaux pour réduire le calcul.

R03 Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference ↗
Benoit Jacob et al. · 2018 · CVPR 2018 · prépublication 2017
Co-concevoir quantification, apprentissage et noyaux entiers.

R04 MCUNet: Tiny Deep Learning on IoT Devices ↗
Ji Lin, Wei-Ming Chen et al. · 2020 · NeurIPS 2020
Chercher ensemble l'architecture du réseau et son moteur d'exécution.

Sources et limites de lecture · 2/6

Articles originaux, prépublications et documentation officielle · état de vérification : 20 octobre 2026.

R05 MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning ↗
Ji Lin, Wei-Ming Chen, Han Cai et al. · 2021 · NeurIPS 2021
Traiter des morceaux de carte de caractéristiques pour réduire le pic mémoire.

R06 On-Device Training Under 256KB Memory ↗
Ji Lin, Ligeng Zhu et al. · 2022 · NeurIPS 2022
Rendre possible une adaptation locale par mises à jour sélectives et co-conception.

R07 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness ↗
Tri Dao, Daniel Y. Fu et al. · 2022 · NeurIPS 2022
Réorganiser le calcul d'attention pour réduire les échanges mémoire sur GPU.

R08 TSMixer: An All-MLP Architecture for Time Series Forecasting ↗
Si-An Chen et al. · 2023 · Transactions on Machine Learning Research, 2023
Mélanger les instants puis les variables avec des perceptrons multicouches.

Sources et limites de lecture · 3/6

Articles originaux, prépublications et documentation officielle · état de vérification : 20 octobre 2026.

R09 AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration ↗
Ji Lin, Jiaming Tang et al. · 2024 · MLSys 2024 · prépublication 2023
Utiliser des statistiques d'activation pour mieux préserver les canaux sensibles.

R10 MobileNetV4 — Universal Models for the Mobile Ecosystem ↗
Danfeng Qin et al. · 2024 · ECCV 2024
Concevoir des blocs et une recherche d'architecture adaptés à plusieurs matériels mobiles.

R11 T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge ↗
Jianyu Wei, Shijie Cao et al. · 2025 · EuroSys 2025 · prépublication 2024
Remplacer certaines opérations de produits mixtes par des tables de recherche adaptées.

R12 SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model ↗
Loubna Ben Allal et al. · 2025 · Prépublication arXiv 2502.02737
La qualité et le mélange des données comptent aussi pour les petits modèles de langage.

Sources et limites de lecture · 4/6

Articles originaux, prépublications et documentation officielle · état de vérification : 20 octobre 2026.

R13 T-MAN: Enabling End-to-End Low-Bit LLM Inference on NPUs via Unified Table Lookup ↗
Jianyu Wei, Qingtao Li et al. · 2025 · Prépublication arXiv · 14 novembre 2025
Adapter tables, tuilage et pipeline aux opérations réellement prises en charge par un NPU.

R14 From Compression to Deployment: Real-Time and Energy-Efficient FastGRNN on Ultra-Constrained Microcontrollers ↗
Emre Can Kizilates · 2026 · Prépublication arXiv · 15 juin 2026
Étudier ensemble compression, activation par table, état récurrent et déploiement physique.

R15 A Framework for Accelerating Transformer Inference on RISC-V for Edge AI ↗
Ajay Kumar M et al. · 2026 · Prépublication arXiv · 6 octobre 2026
Adapter instructions et compilation à des Transformers compacts, principalement encodeurs.

R16 Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations ↗
Apostol Vassilev, Alina Oprea et al. · 2025 · Rapport technique NIST AI 100-2e2025
Distinguer attaques sur données, entrées, modèles et vie privée.

Sources et limites de lecture · 5/6

Articles originaux, prépublications et documentation officielle · état de vérification : 20 octobre 2026.

R17 Tiny Machine Learning: Progress and Futures ↗
Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Song Han · 2023 / 2024 · IEEE Circuits and Systems Magazine 2023 · dépôt arXiv 2024
Vue d'ensemble de la co-conception et du passage de l'inférence à l'adaptation locale.

R18 MLPerf Tiny Benchmark ↗
Colby Banbury, Vijay Janapa Reddi et al. · 2021 · NeurIPS 2021 · Datasets and Benchmarks
Comparer qualité, latence et énergie avec un protocole reproductible.

R19 CMSIS-NN — Efficient neural network kernels for Arm Cortex-M ↗
Arm · Documentation consultée en 2026 · Dépôt officiel de noyaux optimisés
Les performances dépendent de noyaux adaptés à l'ISA et aux formats quantifiés.

R20 LiteRT for Microcontrollers ↗
Google AI Edge · Documentation consultée en 2026 · Documentation officielle du moteur MCU
Déployer un sous-ensemble d'opérateurs avec une allocation adaptée au microcontrôleur.

Sources et limites de lecture · 6/6

Articles originaux, prépublications et documentation officielle · état de vérification : 20 octobre 2026.

R21 **MLPerf Inference: Tiny ↗**

MLCommons · Documentation consultée en 2026 · Méthodologie et résultats officiels

Lire conjointement qualité, scénario de mesure et configuration matérielle.

À lire **MobileNetV4 — Universal Models for the Mobile Ecosystem ↗**

L'écosystème mobile évalué ne couvre pas tous les microcontrôleurs.

À lire **AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration ↗**

Quantifier les poids ne signifie pas quantifier toute la mémoire ni tous les calculs.

À lire **T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge ↗**

Bénéfices dépendants du bit-width, des noyaux, des formes et des matériels comparés.

Lexique · Matériel et mémoire

Des termes à expliquer avec un exemple, pas seulement à mémoriser.

MCU

Microcontrôleur : processeur, mémoire et périphériques réunis pour une application embarquée.

CPU / GPU / NPU

Calcul général / calcul parallèle / accélération spécialisée de réseaux.

Flash

Mémoire non volatile : programme et constantes, selon le placement choisi.

SRAM / RAM

Mémoire de travail volatile. Compter les données simultanément vivantes.

SIMD

Une instruction traite plusieurs valeurs ; les formats et alignements comptent.

DMA

Transfert de données piloté par un contrôleur ; bande passante et synchronisation restent limitées.

ISA

Jeu d'instructions du processeur. Un noyau doit exploiter ce qui est réellement disponible.

Ko / Kio

Ko = 1000 octets ; Kio = 1024 octets. Mio = 1024^2 octets dans ce cours.

Lexique · Modèles et expérimentation

Des termes à expliquer avec un exemple, pas seulement à mémoriser.

Paramètre / activation

Un paramètre est appris ; une activation dépend de l'entrée en cours de calcul.

MAC

Multiplier puis accumuler. Préciser la convention avant de comparer des opérations.

Perte / gradient

La perte mesure une erreur ; le gradient indique comment elle varie avec les paramètres.

PTQ / QAT

Quantification après entraînement / apprentissage tenant compte de la quantification.

Cache KV

Clés et valeurs conservées pour éviter certains recalculs pendant la génération.

Runtime / backend

Moteur qui exécute le graphe / implémentation qui choisit les noyaux de la cible.

Latence / débit

Temps d'une réponse / quantité de réponses par unité de temps. Ils ne se confondent pas.

CI / manifeste

Tests et construction automatisés / fiche qui relie version, conditions et fichiers de l'expérience.

Réponses raisonnées · 1/2

L’argument compte davantage que la lettre choisie.

Question 1 · réponse B · diapositive 8

Le besoin porte sur le service complet. Une solution locale ou hybride doit préciser ce qui reste possible hors connexion, et comment les incertitudes sont gérées.

Question 2 · réponse A · diapositive 18

Le ratio concerne seulement le payload des poids, avant métadonnées. Cache, activations, buffers et noyaux peuvent imposer un autre plafond mémoire ou temps.

Question 3 · réponse C · diapositive 25

Les deux budgets ne se confondent pas. Il faut vérifier la Flash disponible, le placement effectif des constantes, puis le pic SRAM de l’application complète.

Réponses raisonnées · 2/2

L’argument compte davantage que la lettre choisie.

Question 4 · réponse A · diapositive 34

Un résultat local n’établit ni l’effet système ni la portabilité. Le ralentissement est un résultat scientifique à expliquer, pas à cacher.

Question 5 · réponse C · diapositive 43

Une conclusion négative rigoureuse est utile. Elle peut montrer qu’un modèle plus simple convient mieux, ou que l’optimisation cible le mauvais coût.